

Domain: Retail Commerce Operations

Dataset: data/commerce.csv (single dataset)

Global Rules:

- Use ONLY Spark DataFrame transformations (withColumn, filter/where, groupBy, agg, orderBy/sort, limit, distinct, count)
- Do NOT hardcode any dataset values (IDs, region/channel/category/product values, etc.)
- Do NOT use spark.createDataFrame() anywhere (CSV-only assessment)
- df.show() / print() allowed
- Read CSV only from data/

Columns in commerce.csv:

txn_id, order_date, customer_id, region, channel, category, product_id,
quantity, unit_price, discount_rate, returned, ship_date, delivery_date

Implement 20 Pure Functions:

- 1) define_schema() -> StructType
 - Build explicit schema (all nullable) for all 13 columns.
- 2) load_data(spark, path, schema) -> DataFrame
 - spark.read.csv(path, header=True, schema=schema)
- 3) parse_dates(df) -> DataFrame
 - Convert order_date, ship_date, delivery_date to DateType using to_date()
- 4) add_gross_amount(df) -> DataFrame
 - Add gross_amount = quantity * unit_price
- 5) add_net_amount(df) -> DataFrame
 - Add net_amount = gross_amount * (1 - discount_rate)
 - Must compute gross_amount inside or reuse a prior column if present.
- 6) add_delivery_days(df) -> DataFrame
 - Add delivery_days = datediff(delivery_date, ship_date)
- 7) flag_on_time_delivery(df, max_days) -> DataFrame
 - Add is_on_time (boolean): delivery_days <= max_days
- 8) filter_returned_orders(df) -> DataFrame
 - returned == 'Y'
- 9) filter_by_region(df, region) -> DataFrame
 - Filter for provided region
- 10) top_n_customers_by_spend(df, n) -> DataFrame
 - Compute net_amount
 - groupBy(customer_id) sum(net_amount) as total_spend

- order desc by total_spend, tie-break by customer_id asc
- limit(n)

11) revenue_by_category(df) -> DataFrame

- Compute net_amount
- groupBy(category) sum(net_amount) as total_revenue

12) top_category_by_revenue(df) -> str

- Compute net revenue by category
- Return category with max revenue (tie-break by category asc)

13) avg_discount_by_channel(df) -> DataFrame

- groupBy(channel) avg(discount_rate) as avg_discount

14) daily_revenue_trend(df) -> DataFrame

- Compute net_amount
- groupBy(order_date) sum(net_amount) as daily_revenue
- order by order_date asc

15) high_value_orders(df, threshold) -> DataFrame

- Compute net_amount
- Filter net_amount > threshold

16) count_late_deliveries(df, max_days) -> int

- Compute delivery_days
- Count rows where delivery_days > max_days

17) return_rate_by_category(df) -> DataFrame

- For each category:
total_cnt = count(*)
returned_cnt = sum(when(returned=='Y',1).otherwise(0))
return_rate = returned_cnt / total_cnt

18) best_selling_product(df) -> tuple

- groupBy(product_id) sum(quantity) as total_qty
- pick max total_qty (tie-break product_id asc)
- return (product_id, total_qty)

19) list_channels(df) -> List[str]

- distinct channel values
- return sorted python list

20) orders_in_date_range(df, start, end) -> DataFrame

- order_date between start and end (inclusive)
- return filtered df